

ROLLBACK_DESIGN

Helix OTA — Rollback Support Design Document

Document ID: HELOTA-ROLLBACK-001 **Version:** 1.0.1 **Status:** Active **Last Updated:** 2026-03-05 **Constitution Reference:** HelixConstitution v1 §1–§4 **Target Platform:** Android 15 on Orange Pi 5 Max (RK3588) **Precedes:** 1.0.0-MVP System Architecture (HELOTA-ARCH-001) **Depends On:** REST API Specification v1.0.0, Android Client Architecture v1.0.0

Table of Contents

- [1. Rollback Overview](#)
- [2. Android A/B Rollback Mechanism](#)
- [3. Server-Side Rollback](#)
- [4. Client-Side Rollback](#)
- [5. Version History Management](#)
- [6. Data Safety During Rollback](#)
- [7. Rollback API Specification](#)
- [8. Rollback Testing](#)
- [9. Reference Implementation: Go Rollback Engine](#)
- [10. Reference Implementation: Kotlin Android Client Rollback](#)

1. Rollback Overview

1.1 Why Rollback Is Critical for OTA Systems

Over-the-air updates inherently carry risk. Unlike locally-installed software where a user retains physical media or download archives, OTA updates replace system partitions in place, and a failed update can render a device completely non-functional — a state commonly known as “bricking.” For fleet deployments on the Orange Pi 5 Max (RK3588) running Android 15, a failed OTA that cannot be reversed may require physical intervention, on-site technician visits, or RMA returns, all of which carry enormous operational cost.

The Helix OTA system targets enterprise and industrial deployments where device uptime is mission-critical. In such environments, rollback is not a nice-to-have feature — it is a safety requirement on par with artifact signing and dm-verity verification. Without rollback, the update system is a one-way door: once a device accepts an update, there is no recovery path other than manual intervention.

Historical data from large-scale Android deployments demonstrates that 2–5% of OTA updates result in some form of post-install regression, even after extensive testing. The causes range from hardware-specific timing issues (particularly relevant to the RK3588's heterogeneous big.LITTLE core arrangement) to race conditions in vendor HALs that only manifest under specific workload patterns. A rollback mechanism converts these potentially catastrophic failures into transient incidents: the device automatically or manually reverts to its previously-known-good state, reports the failure to the server, and continues operating.

Rollback also provides a critical safety net for phased rollouts. The Helix OTA rollout engine (introduced in 1.0.0) supports incremental percentage-based deployment with auto-advance. Without rollback, discovering a critical bug at the 10% cohort means those devices are stuck on a broken build until a new forward update can be prepared, tested, and deployed. With rollback, those devices can be instantly reverted while the root cause is investigated, collapsing the recovery time from hours or days to minutes.

1.2 Types of Rollback

The Helix OTA 1.0.1 rollback system supports three distinct rollback trigger types, each serving a different failure scenario:

Automatic Rollback (Boot Failure)

When a device successfully applies an OTA payload and reboots into the new slot, but fails to complete the boot sequence within a configurable number of attempts (default: 3), the Android boot control HAL automatically reverts to the previous slot. This is a hardware-level safety mechanism built into the A/B partition layout — it requires no server connectivity and no user interaction. The device detects the failure locally, swaps the active boot slot, and reboots into the previously working system. The Helix OTA client then detects on next startup that a boot-time rollback occurred and reports it to the server.

Server-Triggered Rollback

An operator or automated monitoring system identifies a regression pattern in post-update telemetry (e.g., crash rate increase, health check failure rate exceeding threshold, user complaints) and triggers a rollback via the Helix OTA dashboard or API. This can target a single device, a subset of devices, or an entire rollout cohort. Server-triggered rollback requires the device to be online and reachable, as the rollback command is delivered via the next update check cycle or a push notification.

User-Initiated Rollback

An end-user or on-site technician initiates a rollback through the Android Settings UI (System → System Update → Rollback). This is the rollback of last resort for cases where the device boots successfully but exhibits unacceptable behavior (e.g.,

peripheral malfunction, performance degradation, feature regression). The user-initiated path has a configurable time window (default: 7 days after update) after which the rollback option is no longer available, because by that point the Virtual A/B merge has completed and the previous slot's data is no longer intact.

1.3 Rollback vs. Downgrade

Rollback and downgrade are fundamentally different operations in the Helix OTA system, and conflating them leads to architectural errors:

Property	Rollback	Downgrade
Definition	Reverting to the previously-installed version via A/B slot swap	Installing an older version via a full OTA cycle
Mechanism	Boot control HAL <code>setActiveBootSlot()</code> to the previous slot	Download, verify, and apply a complete OTA package for the older version
Duration	Instant (single reboot)	Full OTA cycle (download + verify + apply + reboot, typically 10–30 min)
Data Safety	Zero data loss — user data partition is untouched	Zero data loss — user data partition is untouched
Semver Constraint	Only to the immediate previous version	Any version with a compatible <code>MinSourceVersion</code>
Merge State	Only possible before Virtual A/B merge completes	Always possible if a compatible artifact exists
App Data Compatibility	Guaranteed (was running this version moments ago)	Not guaranteed (may require forward migration)

The Helix OTA 1.0.1 release supports **rollback only**. Full downgrade support (installing an arbitrary older version) is deferred to a future release because it requires: (1) a complete OTA artifact for the target version, (2) forward database migration scripts, and (3) careful handling of security patch level downgrades which Android forbids by default.

1.4 Data Preservation During Rollback

A core invariant of the Helix OTA rollback system is: **user data is never modified during rollback**. The Android partition layout strictly separates system code from user data:

```
/dev/block/by-name/  
├─ system_a    ← System partition, slot A  
├─ system_b    ← System partition, slot B
```

— vendor_a	← Vendor partition, slot A
— vendor_b	← Vendor partition, slot B
— userdata	← User data (NOT A/B – single copy)
— boot_a	← Kernel + ramdisk, slot A
— boot_b	← Kernel + ramdisk, slot B
— ...	

Rollback operates by changing which slot the bootloader selects at boot time. The userdata partition is not duplicated, not versioned, and not touched during a slot switch. This means all application data, settings, databases, and downloaded files survive a rollback intact.

The one area requiring care is configuration migration: if the new version's first-boot logic migrated a configuration format (e.g., app database schema v2 → v3), a rollback to the old version will encounter the v3 schema. The design mitigates this in Section 6.

2. Android A/B Rollback Mechanism

2.1 How A/B Partition Layout Enables Instant Rollback

The A/B partition layout is the foundational enabler for instant rollback. In a traditional single-slot layout, applying an OTA overwrites the running system in place. If the update fails mid-write, the device is bricked. In an A/B layout, two complete sets of system partitions exist:

Slot A (currently active):

boot_a, system_a, vendor_a, product_a, odm_a

Slot B (inactive, update target):

boot_b, system_b, vendor_b, product_b, odm_b

When an OTA is applied, the update engine writes to the **inactive** slot while the device continues running on the active slot. After the write completes, the bootloader is instructed to boot from the newly-written slot on the next reboot. Rollback is simply a matter of telling the bootloader to go back to the previous slot — no data rewriting is required.

On the RK3588 (Orange Pi 5 Max), the A/B partition layout is defined in the device tree and the GPT (GUID Partition Table). The Rockchip bootloader (U-Boot + trusted firmware) reads the `boot_slot_suffix` property from the Android Boot Control HAL to determine which slot to boot. Changing the active slot is a single HAL call that modifies a persistent boot flag.

2.2 Boot Control HAL

The Android Boot Control HAL (`android.hardware.boot`) provides the programmatic interface for slot management. Helix OTA 1.0.1 uses the following three critical HAL methods:

setActiveBootSlot(slot) — Marks a slot as the active boot target. The bootloader will attempt to boot from this slot on the next reboot. This is the core mechanism for both update (set new slot active) and rollback (set previous slot active).

getActiveBootSlot() — Returns the index of the currently active slot (0 for slot A, 1 for slot B). The client uses this to determine which slot the device is currently running from and which slot would be the rollback target.

markBootSuccessful() — Called after the device has successfully booted into a new slot and completed health checks. This clears the boot retry counter and signals that the current slot is stable. If this method is never called, the boot retry mechanism will eventually trigger an automatic rollback.

The HAL also provides auxiliary methods used by the rollback system:

- **getNumberSlots()** — Returns the number of boot slots (always 2 for A/B devices).
- **getCurrentSlot()** — Returns the slot from which the current boot occurred.
- **isSlotBootable(slot)** — Returns whether a slot contains a valid, bootable image.
- **isSlotMarkedSuccessful(slot)** — Returns whether a slot has been marked as successfully booted.

2.3 Boot Retry Count and Automatic Fallback

Android's boot verification mechanism uses a **retry counter** to detect boot failures and trigger automatic rollback:

1. When **setActiveBootSlot(newSlot)** is called, the boot control HAL sets a retry counter for the new slot (default: 3 attempts).
2. On each boot attempt of the new slot, the bootloader decrements the counter.
3. If **markBootSuccessful()** is called (by the Helix OTA health check service), the counter is cleared and the slot is marked as stable.
4. If the counter reaches zero without **markBootSuccessful()** being called, the bootloader interprets this as a boot failure and automatically falls back to the previous slot.

This mechanism provides **zero-intervention rollback** for boot failures. The device does not need network connectivity, the server does not need to issue a command, and no user interaction is required. The hardware-level fallback is the most reliable rollback path because it operates below the Android framework layer.

The retry count is configurable via the `boot_control` HAL implementation. For Helix OTA on RK3588, we use the default of 3 retries, which provides a reasonable balance between resilience (transient boot issues like slow peripheral initialization get extra chances) and responsiveness (the device recovers within 3 failed boot cycles, typically under 90 seconds total).

2.4 Virtual A/B Snapshot Rollback

Android 15's Virtual A/B mechanism introduces an additional layer of complexity for rollback. Unlike classic A/B where both slots maintain complete partition copies, Virtual A/B uses Copy-on-Write (COW) snapshots to conserve storage:

Update Phase: - The update engine writes new partition data to COW files stored in /data/ota/. - The original slot's partition data is untouched. - snapuserd daemon manages the COW I/O, presenting a virtual block device.

Merge Phase (post-boot): - After the device boots successfully into the new slot and markBootSuccessful() is called, a background merge process begins. - COW data is merged into the underlying partitions of the old slot, which becomes the new inactive slot. - During merge, the device is still protected: if a reboot occurs mid-merge, the merge resumes on next boot.

Rollback Implications: - **Before merge starts** (boot not yet marked successful): Rollback is trivial — simply set the previous slot as active. The COW files are discarded, and the old slot's data is fully intact. - **During merge** (merge in progress): The merge daemon (snapuserd) is suspended, and the device reverts to the old slot. Any partially-merged data in the old slot is discarded because the COW files are still present to serve the current slot's reads. - **After merge completes:** The old slot's partitions have been overwritten with the new version's data. COW files have been deleted. A/B slot swap rollback is **no longer possible** because the previous version no longer exists on either slot. At this point, only a full downgrade (downloading and applying the previous version's OTA package) can revert the device.

This merge completion boundary is why the Helix OTA system enforces a **rollback window**: health checks must pass and markBootSuccessful() must be called before the merge is allowed to proceed. The default health check window is 60 seconds after boot, giving the client time to detect post-boot regressions before the merge commits.

2.5 dm-verity Verification After Rollback

After a rollback (slot switch), the dm-verity (device-mapper verity) integrity verification system must verify the rolled-back slot's partitions. dm-verity uses a Merkle tree to verify the integrity of read-only partitions (system, vendor, product) block-by-block at runtime.

Key considerations for dm-verity after rollback:

1. **Verified Boot State:** Each slot has its own dm-verity state. The rolled-back slot was previously verified when it was the active slot, so its verity tree is already initialized and its verified boot state (GREEN, YELLOW, or ORANGE) was previously established. The bootloader will re-verify the slot on boot using the same keys.
2. **fstab Entries:** The device's fstab file contains verify flags for each mountable partition. These flags are slot-agnostic — they apply to whichever slot is currently active. No fstab modification is needed for rollback.
3. **Key Rotation:** If the new version's OTA included a key rotation (changing the verified boot signing key), rolling back would encounter the old key. Android handles this through the VerifiedBootState mechanism — if the key doesn't match, the boot state is downgraded (e.g., GREEN → YELLOW), but the device still boots. This is a warning, not a blocker.

4. **Care Map:** The OTA package includes a `care_map.pb` protobuf that tells dm-verity which blocks are valid. After rollback, the old slot's care map (stored in the slot's metadata) is used instead. No special handling is required because each slot maintains its own care map.
-

3. Server-Side Rollback

3.1 Rollback API Endpoints

The server-side rollback system exposes three REST API endpoints (detailed in Section 7) for triggering and querying rollbacks:

- `POST /api/v1/devices/{id}/rollback` — Trigger rollback on a single device
- `POST /api/v1/rollouts/{id}/rollback` — Trigger rollback for all devices in a rollout
- `GET /api/v1/devices/{id}/rollback-history` — Query rollback history for a device

All rollback endpoints require admin or operator role authentication and are rate-limited to prevent accidental fleet-wide rollback cascades.

3.2 Rollback Trigger Conditions

The server can automatically detect conditions that warrant a rollback and either trigger it automatically (with admin approval configuration) or surface a recommendation to the operator. The following trigger conditions are monitored:

Failure Rate Threshold: If the failure rate for a rollout cohort exceeds the configurable threshold (default: 5%), the server auto-pauses the rollout and generates a rollback recommendation. The 5% threshold is calculated as: $\text{failed_devices} / (\text{succeeded_devices} + \text{failed_devices})$ among devices that have reached a terminal state.

Boot Loop Detection: If a device reports consecutive failed boot attempts without a successful `markBootSuccessful` telemetry event within the expected window (default: 10 minutes post-reboot), the server flags the device for rollback.

Health Check Failure Pattern: If more than 3 devices in the same rollout cohort report health check failures (post-update verification failures) within a 15-minute window, the server generates a fleet-wide rollback alert.

Anomaly Detection: The existing telemetry anomaly detection system (1.0.0) is extended with rollback-specific rules: - Crash rate increase > 300% compared to pre-update baseline - Device offline rate increase > 200% in the 24 hours following an update - Any device reporting `INSTALL_ENGINE_ERROR` with error code indicating partition corruption

3.3 Fleet-Wide Rollback Execution

When a fleet-wide rollback is triggered (via `POST /api/v1/rollouts/{id}/rollback`), the server executes the following sequence:

1. **Pause the rollout** — Set rollout status to `ROLLING_BACK`, preventing any new devices from starting the update.
2. **Enumerate affected devices** — Query all devices that received the update via this rollout and have not yet been rolled back.
3. **Classify devices** — Partition the device list into three categories:
 - **Pre-merge devices** (health check not yet passed): Can be rolled back via slot swap.
 - **Post-merge devices** (merge completed): Cannot be rolled back via slot swap; require a forward update to the previous version.
 - **Offline devices**: Queued for rollback on next check-in.
4. **Issue rollback commands** — For pre-merge devices, the server injects a `rollback_pending` flag into the device's update check response. For post-merge devices, the server creates a "recovery rollout" targeting the previous version.
5. **Monitor rollback progress** — Track rollback completion via telemetry. Alert operators if rollback does not complete within the expected window (default: 30 minutes for online devices, 24 hours for offline devices).

3.4 Rollback Rollout (Gradual or Instant)

Server-triggered rollbacks can be executed in two modes:

Instant Rollback: All targeted devices receive the rollback command simultaneously. This is appropriate for critical failures (boot loops, data corruption risk) where rapid recovery outweighs the risk of a large simultaneous reboot event.

Gradual Rollback: Devices are rolled back in waves (default: 25% per wave, 5-minute intervals). This is appropriate for non-critical regressions (performance degradation, minor feature breakage) where the operator wants to observe the rollback's impact on fleet stability before committing to a full rollback.

The rollback mode is configurable per rollback request via the `strategy` field:

```
{
  "strategy": "gradual",
  "wave_percentage": 25,
  "wave_interval_seconds": 300,
  "reason": "Performance regression: boot time increased by 40%"
}
```

3.5 Audit Trail for Rollbacks

Every rollback action, regardless of trigger type, generates an immutable audit record stored in the `rollback_audit_log` database table:

```
CREATE TABLE rollback_audit_log (
  id          VARCHAR(32) PRIMARY KEY,
  device_id   VARCHAR(32) NOT NULL REFERENCES devices(id),
```

```

rollout_id      VARCHAR(32) REFERENCES rollouts(id),
from_version    VARCHAR(32) NOT NULL,
to_version      VARCHAR(32) NOT NULL,
trigger_type    VARCHAR(32) NOT NULL, -- 'automatic',
               'server', 'user'
trigger_reason  TEXT NOT NULL,
triggered_by    VARCHAR(32),          -- user_id for server/
               user triggers, NULL for automatic
status          VARCHAR(32) NOT NULL, -- 'pending',
               'in_progress', 'completed', 'failed'
created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
completed_at    TIMESTAMPTZ,
metadata        JSONB DEFAULT '{}
);

```

Audit records are never deleted and are append-only. They support compliance requirements for regulated deployments where every system change must be traceable.

4. Client-Side Rollback

4.1 Detecting the Need for Rollback

The Helix OTA Android client detects the need for rollback through three independent mechanisms:

Boot Failure Detection: After an OTA update and reboot, if the device fails to boot and the boot control HAL's retry counter triggers an automatic fallback, the client detects this on the next successful boot by comparing the active slot against the expected slot. If the device is now running from the slot that was active before the update, a boot-time rollback occurred.

Health Check Failure: After a successful boot into the new slot, the Helix OTA HealthCheckService runs a battery of post-update verification checks. If any critical check fails, the service initiates a rollback before the merge phase begins.

Server Command: During a routine update check, the server may include a `rollback_command` in the response. The client processes this command and initiates the rollback sequence.

4.2 Automatic Rollback on Boot Failure

The boot failure rollback flow is entirely handled by the Android boot control HAL and requires no client code execution during the failure. The client's role is to detect and report after the fact:

1. Update is applied, `setActiveBootSlot(newSlot)` is called, device reboots.
2. New slot fails to boot (kernel panic, critical service crash, init loop).
3. Bootloader decrements retry count on each failed attempt.
4. After 3 failed attempts, bootloader switches to the previous slot.
5. Device boots successfully into the old (pre-update) slot.

6. `BootCompletedReceiver` in the Helix OTA client fires.
7. The client's `RollbackDetectionService` checks:
 - Current slot $\neq$ expected post-update slot $\rightarrow$ boot rollback detected.
 - Previous update record exists with status `REBOOTING`.
8. Client updates the update record status to `ROLLED_BACK`.
9. Client reports the rollback to the Helix OTA server via `POST /api/v1/telemetry`.
10. Client does NOT call `markBootSuccessful()` — the new slot remains unmarked, preventing future boot attempts into it.

4.3 Health Check Service: Post-Update Verification

The `HealthCheckService` runs immediately after the device boots into a new slot and before `markBootSuccessful()` is called. This is the critical window where software-level rollback decisions are made. The service executes the following checks:

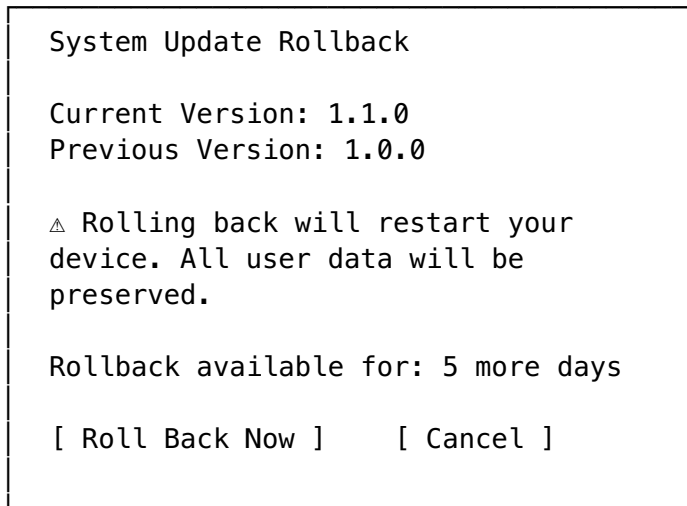
Check	Description	Failure Action
Boot Time	Measures time from bootloader to <code>BOOT_COMPLETED</code> broadcast. Compares against pre-update baseline (stored in shared prefs). Threshold: $>150\%$ of baseline.	Rollback
Critical Service Health	Checks that <code>system_server</code> , <code>surfaceflinger</code> , and <code>vold</code> are running and responsive. Uses <code>ServiceManager.getService()</code> + health check ping.	Rollback
Network Connectivity	Verifies that WiFi or Ethernet is functional (can reach the Helix OTA server). Retries 3 times with 5-second intervals.	Warn only
Storage Access	Verifies that <code>/data</code> and <code>/sdcard</code> are mounted and writable. Creates and deletes a test file.	Rollback
Peripheral Status	Checks that RK3588-specific peripherals (GPU, NPU, VPU) are initialized. Reads from <code>/sys/class/devfreq/</code> and <code>/dev/mali0</code> .	Warn only
App Launch Test	Launches a test Activity from the Helix OTA app and verifies it renders within 5 seconds.	Warn only

The health check has a **maximum execution time of 60 seconds**. If the check does not complete within this window (e.g., because the device is stuck in ANR), the `WatchdogTimer` fires and triggers a rollback.

Health check results are stored locally and reported to the server. Only checks marked “Rollback” in the failure action column trigger an automatic rollback. “Warn only” checks generate a telemetry event but allow the update to proceed.

4.4 User-Initiated Rollback from Settings

The user-initiated rollback is exposed through the Android Settings app via a Settings Provider integration. The Helix OTA client registers a Settings fragment at Settings → System → System Update → Rollback:



The rollback option is only available when: 1. The device has a previously active slot with a valid, bootable image (`isSlotBootable(previousSlot) == true`). 2. The Virtual A/B merge has not yet completed (checked via `BootControlHelper.isMergePending()`). 3. The rollback window has not expired (default: 7 days since the update was applied). 4. No other update or rollback is currently in progress.

When the user taps “Roll Back Now,” the system: 1. Displays a confirmation dialog with the version numbers. 2. On confirmation, calls `setActiveBootSlot(previousSlot)`. 3. Reboots the device immediately. 4. On next boot into the previous version, the client reports the user-initiated rollback to the server.

4.5 Rollback State Reporting

After any rollback (automatic, server-triggered, or user-initiated), the client reports the rollback to the server using the existing telemetry endpoint with a dedicated event type:

```
{
  "device_id": "dev_01HWIDGET001",
  "event_type": "ROLLBACK",
  "status": "completed",
  "error_code": null,
  "metadata": {
    "trigger_type": "automatic",
    "from_version": "1.1.0",
    "to_version": "1.0.0",
    "from_slot": 1,
    "to_slot": 0,
    "health_check_results": {
```

```

        "boot_time_ms": 45000,
        "boot_time_baseline_ms": 18000,
        "boot_time_exceeded": true,
        "critical_services_ok": false,
        "system_server_anr": true
    },
    "rollback_duration_ms": 3200
},
"timestamp": "2026-03-05T14:30:00Z"
}

```

The server updates the device's `current_version` and `slot_suffix` fields and creates a rollback audit record.

5. Version History Management

5.1 How Many Versions to Keep Available for Rollback

The A/B partition layout inherently supports rolling back to exactly **one** previous version — the version on the inactive slot. This is a hard constraint of the A/B architecture: there are only two slots, and the inactive slot always contains the version that was running before the most recent update.

Helix OTA 1.0.1 does not extend this beyond the single-previous-version constraint. Supporting multi-version rollback (e.g., rolling back from 1.3.0 to 1.1.0, skipping 1.2.0) would require either:

- **Additional partition slots** (A/B/C layout): Not supported by the RK3588 bootloader or Android's boot control HAL.
- **Storing previous version images in /data**: Feasible but requires significant storage (1–3 GB per image) and a custom update engine to apply them. This is deferred to a future release.
- **Server-side downgrade**: Downloading and applying the previous version's OTA package. This is technically a downgrade, not a rollback, and is also deferred.

The practical impact of the single-version constraint is minimal because: 1. Enterprise deployments typically discover regressions within hours, well before a second update is applied. 2. The health check window (60 seconds post-boot) catches most regressions before the merge commits. 3. If a second update is applied before a regression is discovered, the server can still create a recovery rollout using the old version's OTA artifact.

5.2 Partition Space Management for Rollback Images

The inactive slot's partitions consume space equal to the active slot's partitions. On the RK3588 with a 16 GB eMMC, the typical allocation is:

Partition	Size (per slot)	Purpose
boot	64 MB	Kernel + ramdisk

Partition	Size (per slot)	Purpose
system	2.5 GB	Android framework
vendor	800 MB	Vendor HALs
product	400 MB	Product-specific apps
odm	200 MB	ODM customizations
Total per slot	~4 GB	

The A/B layout doubles this to ~8 GB, leaving ~8 GB for userdata, cache, and other partitions. This is a fixed cost of the A/B architecture and is not impacted by the rollback feature.

5.3 Super Partition Snapshot Management for Virtual A/B

Virtual A/B reduces the storage overhead by using COW snapshots instead of full duplicate partitions. The super partition contains both slot's logical partitions, and during an update, COW files are created in `/data/ota/`:

```
/data/ota/
├─ system_b.cow      (up to system_b partition size, typically
2.5 GB)
├─ vendor_b.cow      (up to 800 MB)
├─ product_b.cow     (up to 400 MB)
└─ odm_b.cow         (up to 200 MB)
```

The COW files are temporary — they exist only during the update-to-merge window. After merge completes, they are deleted. The maximum additional storage consumed by COW files equals the total size of the inactive slot's partitions (~4 GB).

For rollback, the COW files must be preserved until the health check passes. The Helix OTA client prevents premature COW deletion by delaying `markBootSuccessful()` until health checks complete. If a rollback occurs, the COW files are simply discarded (deleted) — they are no longer needed because the device is reverting to the base partition data.

5.4 Garbage Collection of Old Rollback Images

After a successful update (health checks pass, `markBootSuccessful()` called, merge completes), the following cleanup occurs:

1. **COW file deletion:** The `snapperd` daemon deletes all COW files in `/data/ota/` after merge completes. This is handled automatically by the Android framework.
2. **OTA zip deletion:** The Helix OTA client deletes the downloaded OTA zip file from `/data/ota/` after the merge completes. This frees 1–3 GB.
3. **Extracted payload deletion:** The extracted `payload.bin` and `payload_properties.txt` files are deleted after `update_engine` reports completion.

4. **Stale update records:** The client's Room database retains update records for 30 days, then purges records older than the retention period. This prevents unbounded database growth.
 5. **Server-side artifact cleanup:** The server retains all artifact versions indefinitely (they are needed for potential recovery rollouts). Artifact binary files in S3/MinIO are only deleted when an admin explicitly deletes the artifact via the API, and only if the artifact is not referenced by any active or completed rollout.
-

6. Data Safety During Rollback

6.1 User Data Partition Is Never Touched During Rollback

This is the cardinal rule of the Helix OTA rollback system, and it is enforced at the architecture level:

- The rollback mechanism operates exclusively on the boot slot selection. It changes **which partitions the bootloader reads**, not the contents of any partition.
- The userdata partition is not part of the A/B slot scheme. It is a single, shared partition that is mounted identically regardless of which slot is active.
- The rollback API endpoints and client code never issue write commands to userdata. The `setActiveBootSlot()` HAL call modifies only the boot control metadata (stored in a dedicated `boot_ab` or `misc` partition), not user data.

This invariant is verified by the test suite (Section 8) through a pre/post rollback data integrity check: a known test file is written to `/sdcard/` before the update, and its contents are verified after rollback.

6.2 /data Preservation Across Rollbacks

Beyond the userdata partition (mounted at `/data`), several system directories on `/data` must survive a rollback:

Path	Purpose	Rollback Impact
<code>/data/data/</code>	App internal storage	Preserved — mounted from userdata
<code>/data/media/</code>	User media files	Preserved — mounted from userdata
<code>/data/system/</code>	System configuration	Preserved, but version-specific files may cause warnings
<code>/data/dalvik-cache/</code>	Optimized DEX files	Regenerated on first boot — old cache is discarded
<code>/data/ota/</code>	OTA temporary files	May contain COW files; cleaned up per Section 5.4
	Boot control state	

Path	Purpose	Rollback Impact
/data/ misc/ bootctl/		Modified by rollback (slot change) — this is the intended effect

The Dalvik cache regeneration after rollback may cause the first boot to be slightly slower (typically 10–30 seconds additional) as the runtime re-optimizes application DEX files for the previous version’s system framework.

6.3 App Data Compatibility Between Versions

When a device rolls back from version N+1 to version N, applications that were running on N+1 may have upgraded their local data formats. For example:

- An app’s SQLite database may have been migrated from schema version 3 to schema version 4 during the N+1 first boot.
- SharedPreferences may contain keys that were added in N+1 and are unknown to version N’s code.
- The system settings database may have new entries added by N+1’s system services.

The Helix OTA system handles this through a **forward-compatible migration** design pattern that all Helix-managed apps must follow:

1. **Database schemas are additive only** — New columns and tables are added; existing columns are never removed or renamed. Version N’s code ignores columns it doesn’t recognize.
2. **SharedPrefs keys are namespaced by version** — Keys added in N+1 are prefixed with `v_next_`. Version N’s code does not read these keys.
3. **Settings entries use typed accessors** — Settings reads use `getInt(key, defaultValue)` with sensible defaults, so missing keys from N+1 are handled gracefully.

For third-party apps not under Helix’s control, the risk is real but acceptable: most Android apps are designed to handle data format variability (because they must support installs on multiple Android versions). The worst case is that an app crashes on first launch after rollback and the user force-stops and reopens it, at which point the app’s own error recovery logic handles the format mismatch.

6.4 Configuration Migration During Rollback

System-level configuration migration during rollback is handled by the `RollbackConfigMigrator` component in the Helix OTA client:

1. **Before update:** The client snapshots key system configuration files to `/data/helix_ota/rollback_config_snapshot/`. This includes:
 - `/data/system/packages.xml` (package manager state)
 - `/data/system/sysconfig/` (system configuration)
 - `/data/misc/wifi/` (WiFi configuration)
 - `/data/system_ce/0/` (credential storage)

2. **After rollback:** The client compares the current configuration with the snapshot. If significant schema differences are detected (e.g., `packages.xml` format version changed), the client:
- Logs a warning to the telemetry system.
 - Triggers a `PackageManager` reconciliation by clearing the `dalvik-cache` and forcing a rescan.
 - In extreme cases, performs a selective config restoration from the snapshot.

This migration is conservative: the default behavior is to accept the current configuration as-is and rely on Android's built-in backward compatibility. Config restoration is a last resort triggered only when the configuration format version regressed more than one major version.

7. Rollback API Specification

7.1 POST `/api/v1/devices/{id}/rollback`

Trigger a rollback on a single device, reverting it to the previous firmware version via A/B slot swap.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 10 requests/minute per device

Path Parameters:

Parameter	Type	Required	Description
<code>id</code>	string	Yes	Device identifier

Request Body:

```
{
  "target_version": "1.0.0",
  "reason": "Post-update regression: boot time increased 150%",
  "strategy": "instant"
}
```

Field	Type	Required	Validation	Description
<code>target_version</code>	string	No	Semver format	Specific version to roll back to (defaults to previous version)
<code>reason</code>	string	Yes	1–512 chars	Reason for rollback (audit log)
<code>strategy</code>	string	No	One of: instant, gradual	Rollback execution strategy (default: instant)

Go struct:

```
type DeviceRollbackRequest struct {
    TargetVersion string `json:"target_version,omitempty"
        validate:"omitempty,semver"`
    Reason        string `json:"reason"
        validate:"required,min=1,max=512"`
    Strategy      string `json:"strategy,omitempty"
        validate:"omitempty,oneof=instant gradual"`
}
```

Success Response (202 Accepted):

```
{
    "rollback_id": "rbk_01HROLLBACK1",
    "device_id": "dev_01HWIDGET001",
    "from_version": "1.1.0",
    "target_version": "1.0.0",
    "strategy": "instant",
    "status": "pending",
    "merge_completed": false,
    "rollback_possible": true,
    "estimated_duration_seconds": 60,
    "created_at": "2026-03-05T14:30:00Z"
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Field validation failure
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot trigger rollback
404	DEVICE_NOT_FOUND	Device does not exist
409	NO_ROLLBACK_AVAILABLE	Device has no previous version to roll back to (merge already completed)
409	ROLLBACK_ALREADY_IN_PROGRESS	A rollback is already pending for this device
422	TARGET_VERSION_MISMATCH	The specified target_version does not match the inactive slot's version

7.2 POST /api/v1/rollouts/{id}/rollback

Trigger a rollback for all devices that received an update through a specific rollout.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 3 requests/minute per rollout

Path Parameters:

Parameter	Type	Required	Description
id	string	Yes	Rollout identifier

Request Body:

```
{
  "reason":
    "Critical regression: NPU initialization failure on 8% of
    devices",
  "strategy": "gradual",
  "wave_percentage": 25,
  "wave_interval_seconds": 300,
  "include_post_merge": false,
  "dry_run": false
}
```

Field	Type	Required	Validation	Description
reason	string	Yes	1–512 chars	Reason for fleet rollback (audit log)
strategy	string	No	One of: instant, gradual	Execution strategy (default: gradual)
wave_percentage	integer	No	1–100	Percentage of devices per wave (default: 25, only for gradual strategy)
wave_interval_seconds	integer	No	60–3600	Seconds between waves (default: 300)
include_post_merge	boolean	No	—	Include devices where merge has completed (requires downgrade artifact, default: false)
dry_run	boolean	No	—	Simulate the rollback without

Field	Type	Required	Validation	Description
				executing it (default: false)

Go struct:

```

type RolloutRollbackRequest struct {
    Reason      string `json:"reason"
        validate:"required,min=1,max=512"`
    Strategy    string `json:"strategy,omitempty"
        validate:"omitempty,oneof=instant gradual"`
    WavePercentage int    `json:"wave_percentage,omitempty"
        validate:"omitempty,min=1,max=100"`
    WaveIntervalSeconds int
        `json:"wave_interval_seconds,omitempty"
        validate:"omitempty,min=60,max=3600"`
    IncludePostMerge bool
        `json:"include_post_merge,omitempty"`
    DryRun      bool    `json:"dry_run,omitempty"`
}

```

Success Response (202 Accepted):

```

{
  "rollback_id": "rbk_01HFLEETROLL1",
  "rollout_id": "rol_01HROLL001",
  "from_version": "1.1.0",
  "target_version": "1.0.0",
  "strategy": "gradual",
  "total_devices": 482,
  "pre_merge_devices": 387,
  "post_merge_devices": 62,
  "offline_devices": 33,
  "excluded_post_merge": 62,
  "waves": [
    { "wave": 1, "device_count": 97, "starts_at":
      "2026-03-05T14:30:00Z" },
    { "wave": 2, "device_count": 97, "starts_at":
      "2026-03-05T14:35:00Z" },
    { "wave": 3, "device_count": 97, "starts_at":
      "2026-03-05T14:40:00Z" },
    { "wave": 4, "device_count": 96, "starts_at":
      "2026-03-05T14:45:00Z" }
  ],
  "status": "pending",
  "created_at": "2026-03-05T14:30:00Z"
}

```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Field validation failure

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot trigger fleet rollback
404	ROLLOUT_NOT_FOUND	Rollout does not exist
409	ROLLOUT_ALREADY_ROLLING_BACK	A rollback is already in progress for this rollout
409	NO_ROLLBACK_AVAILABLE	All devices in this rollout have completed merge

7.3 GET /api/v1/devices/{id}/rollback-history

Retrieve the rollback history for a specific device.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 120 requests/minute per user

Path Parameters:

Parameter	Type	Required	Description
id	string	Yes	Device identifier

Query Parameters:

Parameter	Type	Required	Description
limit	integer	No	Items per page (1–500, default 50)
cursor	string	No	Pagination cursor

Success Response (200):

```
{
  "data": [
    {
      "rollback_id": "rbk_01HROLLBACK1",
      "device_id": "dev_01HWIDGET001",
      "rollout_id": "rol_01HROLL001",
      "from_version": "1.1.0",
      "to_version": "1.0.0",
      "trigger_type": "automatic",
      "trigger_reason":
        "Boot time exceeded 150% of baseline after update",
      "triggered_by": null,
      "status": "completed",
      "health_check_results": {
        "boot_time_ms": 45000,
        "boot_time_baseline_ms": 18000,
        "critical_services_ok": false
      },
    },
  ],
}
```

```

        "created_at": "2026-03-05T14:30:00Z",
        "completed_at": "2026-03-05T14:31:05Z"
    }
],
"pagination": {
    "total_count": 1,
    "has_more": false,
    "next_cursor": "",
    "limit": 50
}
}

```

Go struct:

```

type RollbackHistoryEntry struct {
    RollbackID      string    `json:"rollback_id"`
    DeviceID        string    `json:"device_id"`
    RolloutID       *string   `json:"rollout_id"`
    FromVersion     string    `json:"from_version"`
    ToVersion       string    `json:"to_version"`
    TriggerType     string    `json:"trigger_type"`
    TriggerReason   string    `json:"trigger_reason"`
    TriggeredBy     *string   `json:"triggered_by"`
    Status          string    `json:"status"`
    HealthCheckResults *RollbackHealthChecks
    `json:"health_check_results,omitempty"`
    CreatedAt       time.Time `json:"created_at"`
    CompletedAt     *time.Time `json:"completed_at"`
}

type RollbackHealthChecks struct {
    BootTimeMs      int    `json:"boot_time_ms"`
    BootTimeBaselineMs int    `json:"boot_time_baseline_ms"`
    CriticalServicesOk bool   `json:"critical_services_ok"`
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	DEVICE_NOT_FOUND	Device does not exist

8. Rollback Testing

8.1 Test Scenarios for Each Rollback Type

The following test matrix covers all rollback trigger types and their combinations:

Test ID	Rollback Type	Scenario	Expected Outcome
RB-001	Automatic	Boot failure (kernel panic in new slot)	Bootloader falls back after 3 retries; client detects and reports rollback
RB-002	Automatic	Boot failure (init loop in new slot)	Bootloader falls back; client detects rollback on successful boot
RB-003	Server	Single device rollback via API	Device receives rollback command on next check-in; slot is swapped; device reboots into previous version
RB-004	Server	Fleet-wide rollback via rollout API	All targeted devices receive rollback in waves; telemetry tracks completion
RB-005	User	User-initiated rollback from Settings	Slot is swapped immediately; device reboots; rollback reported to server
RB-006	User	Rollback option disabled after merge	Settings UI shows “Rollback not available”; no rollback action possible
RB-007	User	Rollback option expired (7-day window)	Settings UI shows “Rollback period expired”; no rollback action possible

8.2 Boot Failure Simulation

Boot failure is simulated using a specially-crafted test OTA package that includes a postinst script which writes a kernel panic trigger:

```
#!/system/bin/sh
# Test: Simulate boot failure by writing invalid kernel command
#       line
echo "panic=1 init=/system/bin/false" > /proc/cmdline_override 2>/dev/null || true
# Alternative: disable critical service
stop zygote
```

The test procedure:

1. Build a test OTA package with the panic-triggering postinst script.
2. Upload the test artifact to the Helix OTA server.
3. Create a rollout targeting the test device group.
4. Wait for the device to download, verify, and apply the update.
5. Device reboots into the new slot and fails to boot.
6. After 3 retries, bootloader falls back to the previous slot.
7. Verify: Device is running the previous version.

8. Verify: Client reports ROLLBACK telemetry event with `trigger_type:` "automatic".
9. Verify: Server audit log records the automatic rollback.

8.3 Health Check Failure Simulation

Health check failure is simulated by deploying an OTA that boots successfully but triggers a health check violation:

```
#!/system/bin/sh
# Test: Simulate health check failure by delaying boot beyond
        threshold
sleep 45 # Push boot time to >150% of baseline
```

The test procedure:

1. Build a test OTA with a boot-delaying init script.
2. Upload, create rollout, and deploy to test device.
3. Device boots into new slot, but HealthCheckService detects boot time exceeds threshold.
4. HealthCheckService triggers automatic rollback before `markBootSuccessful()` is called.
5. Device reboots into previous slot.
6. Verify: Device is running previous version.
7. Verify: Health check results are included in the rollback telemetry event.

8.4 Concurrent Rollback During Active Update

This test verifies that the system correctly handles the edge case where a rollback command arrives while an update is in progress:

State	Rollback Action	Expected Behavior
Downloading	Server triggers rollback	Download is cancelled; slot is not swapped; device remains on current version
Verifying	Server triggers rollback	Verification is cancelled; device remains on current version
Installing (update_engine running)	Server triggers rollback	update_engine is cancelled via <code>cancel()</code> ; slot is not swapped; device remains on current version
Rebooting (slot already switched)	Server triggers rollback	Rollback proceeds normally — the new slot is the one being rolled back from
Committing (merge in progress)	Server triggers rollback	Merge is suspended; device reverts to previous slot

The test verifies that under no circumstances does a concurrent rollback + update result in data loss or an unbootable state.

8.5 Power Failure During Rollback

Power failure during rollback is the most dangerous edge case because it can occur at any point in the rollback sequence. The following power failure points are tested:

1. **Before `setActiveBootSlot()`:** The slot hasn't been changed. On power recovery, the device boots into the current (new) slot as if the rollback never started. The client retries the rollback.
2. **During `setActiveBootSlot()`:** The boot control HAL writes the slot flag atomically (single block write to the misc partition). Either the write completes (rollback succeeds) or it doesn't (device boots into the new slot). No partial state is possible.
3. **After `setActiveBootSlot()`, before reboot:** The slot flag is set. On power recovery, the device boots into the previous (rolled-back) slot. This is the correct outcome.
4. **During reboot (bootloader executing):** The bootloader reads the slot flag and boots from the specified slot. Power failure during boot simply results in a retry on the next power cycle.
5. **During Virtual A/B merge suspension:** If the merge was in progress when the rollback was triggered, the `snapperd` daemon is suspended. If power fails during suspension, on recovery, the bootloader detects the suspended merge and either resumes it or cancels it depending on the merge state. In all cases, the device boots from a consistent slot.

The critical invariant verified by these tests is: **at no point during a rollback does a power failure result in data corruption or an unbootable device**. The A/B architecture's atomic slot switching guarantees this.

9. Reference Implementation: Go Rollback Engine

The following Go code implements the server-side rollback engine that integrates with the existing Helix OTA server architecture.

```
package rollback
```

```
import (  
    "context"  
    "fmt"  
    "time"  
  
    "github.com/helix-ota/internal/eventbus"  
    "github.com/helix-ota/internal/storage"  
    "github.com/vasic-digital/cache"  
    "github.com/vasic-digital/concurrency"  
)
```

```
//  
// Domain Types
```

```
//
type RollbackTriggerType string

const (
    TriggerAutomatic RollbackTriggerType = "automatic"
    TriggerServer     RollbackTriggerType = "server"
    TriggerUser       RollbackTriggerType = "user"
)

type RollbackStatus string

const (
    RollbackPending     RollbackStatus = "pending"
    RollbackInProgress RollbackStatus = "in_progress"
    RollbackCompleted   RollbackStatus = "completed"
    RollbackFailed      RollbackStatus = "failed"
)

type RollbackStrategy string

const (
    StrategyInstant RollbackStrategy = "instant"
    StrategyGradual RollbackStrategy = "gradual"
)

// RollbackRecord represents a single rollback operation.
type RollbackRecord struct {
    ID string `json:"id" db:"id"`
    DeviceID string `json:"device_id" db:"device_id"`
    RolloutID *string `json:"rollout_id" db:"rollout_id"`
    FromVersion string `json:"from_version" db:"from_version"`
    ToVersion string `json:"to_version" db:"to_version"`
    TriggerType RollbackTriggerType `json:"trigger_type" db:"trigger_type"`
    TriggerReason string `json:"trigger_reason" db:"trigger_reason"`
    TriggeredBy *string `json:"triggered_by" db:"triggered_by"`
    Strategy RollbackStrategy `json:"strategy" db:"strategy"`
    Status RollbackStatus `json:"status" db:"status"`
    MergeCompleted bool `json:"merge_completed" db:"merge_completed"`
    HealthCheckData map[string]interface{} `json:"health_check_data" db:"health_check_data"`
    CreatedAt time.Time `json:"created_at" db:"created_at"`
    CompletedAt *time.Time `json:"completed_at" db:"completed_at"`
}
```

```

}

// DeviceRollbackRequest is the API request for a single-device
// rollback.
type DeviceRollbackRequest struct {
    TargetVersion string `json:"target_version,omitempty"
        validate:"omitempty,semver"`
    Reason        string `json:"reason"
        validate:"required,min=1,max=512"`
    Strategy      string `json:"strategy,omitempty"
        validate:"omitempty,oneof=instant gradual"`
}

// RolloutRollbackRequest is the API request for a fleet-wide
// rollback.
type RolloutRollbackRequest struct {
    Reason        string `json:"reason"
        validate:"required,min=1,max=512"`
    Strategy      string `json:"strategy,omitempty"
        validate:"omitempty,oneof=instant gradual"`
    WavePercentage int    `json:"wave_percentage,omitempty"
        validate:"omitempty,min=1,max=100"`
    WaveIntervalSeconds int `json:"wave_interval_seconds,omitempty"
        validate:"omitempty,min=60,max=3600"`
    IncludePostMerge bool `json:"include_post_merge,omitempty"`
    DryRun         bool `json:"dry_run,omitempty"`
}

// RollbackWave represents a single wave in a gradual fleet
// rollback.
type RollbackWave struct {
    WaveNumber int    `json:"wave"`
    DeviceCount int    `json:"device_count"`
    StartsAt    time.Time `json:"starts_at"`
}

// RolloutRollbackResponse is the API response for fleet rollback.
type RolloutRollbackResponse struct {
    RollbackID      string `json:"rollback_id"`
    RolloutID       string `json:"rollout_id"`
    FromVersion     string `json:"from_version"`
    TargetVersion   string `json:"target_version"`
    Strategy        string `json:"strategy"`
    TotalDevices    int    `json:"total_devices"`
    PreMergeDevices int    `json:"pre_merge_devices"`
    PostMergeDevices int    `json:"post_merge_devices"`
    OfflineDevices  int    `json:"offline_devices"`
    ExcludedPostMerge int    `json:"excluded_post_merge"`
    Waves           []RollbackWave `json:"waves,omitempty"`
    Status          string `json:"status"`
    CreatedAt       time.Time `json:"created_at"`
}

```

```

// _____
// Repository Interface
// _____

type RollbackRepository interface {
    Create(ctx context.Context, record *RollbackRecord) error
    GetByID(ctx context.Context, id string) (*RollbackRecord,
        error)
    GetByDeviceID(ctx context.Context, deviceID string, limit
        int, cursor string) ([]RollbackRecord, string, error)
    GetActiveByDeviceID(ctx context.Context, deviceID string)
        (*RollbackRecord, error)
    UpdateStatus(ctx context.Context, id string, status
        RollbackStatus, completedAt *time.Time) error
    ListByRolloutID(ctx context.Context, rolloutID string)
        ([]RollbackRecord, error)
}

type DeviceRepository interface {
    GetByID(ctx context.Context, id string) (*Device, error)
    UpdateVersion(ctx context.Context, id string, version string,
        slot string) error
    ListByRolloutID(ctx context.Context, rolloutID string)
        ([]Device, error)
}

type RolloutRepository interface {
    GetByID(ctx context.Context, id string) (*Rollout, error)
    UpdateStatus(ctx context.Context, id string, status string)
        error
}

type Device struct {
    ID            string    `json:"id"`
    CurrentVersion string    `json:"current_version"`
    PreviousVersion *string   `json:"previous_version"`
    SlotSuffix     string    `json:"slot_suffix"`
    MergeCompleted bool      `json:"merge_completed"`
    Status         string    `json:"status"`
    Group          string    `json:"device_group"`
}

type Rollout struct {
    ID            string    `json:"id"`
    ArtifactID     string    `json:"artifact_id"`
    DeviceGroup    string    `json:"device_group"`
    CurrentPercentage int      `json:"current_percentage"`
    Status         string    `json:"status"`
    TargetVersion  string    `json:"target_version"`
}

// _____
// Rollback Service

```

```
// -----

// RollbackService manages the full lifecycle of rollback
// operations.
type RollbackService struct {
    rollbackRepo RollbackRepository
    deviceRepo   DeviceRepository
    rolloutRepo  RolloutRepository
    cache        cache.Provider
    events       eventbus.Publisher
    workers      *concurrency.Pool
    idGenerator  func(prefix string) string
}

// NewRollbackService creates a new RollbackService.
func NewRollbackService(
    rollbackRepo RollbackRepository,
    deviceRepo DeviceRepository,
    rolloutRepo RolloutRepository,
    cache cache.Provider,
    events eventbus.Publisher,
    workers *concurrency.Pool,
) *RollbackService {
    return &RollbackService{
        rollbackRepo: rollbackRepo,
        deviceRepo:   deviceRepo,
        rolloutRepo:  rolloutRepo,
        cache:        cache,
        events:       events,
        workers:      workers,
        idGenerator:  generateID,
    }
}

// RollbackDevice initiates a rollback for a single device.
func (s *RollbackService) RollbackDevice(
    ctx context.Context,
    deviceID string,
    req DeviceRollbackRequest,
    triggeredBy *string,
) (*RollbackRecord, error) {
    // 1. Load device
    device, err := s.deviceRepo.GetByID(ctx, deviceID)
    if err != nil {
        return nil, fmt.Errorf("device lookup: %w", err)
    }

    // 2. Check for existing rollback in progress
    existing, _ := s.rollbackRepo.GetActiveByDeviceID(ctx,
        deviceID)
    if existing != nil {
        return nil, ErrRollbackAlreadyInProgress{DeviceID:
            deviceID}
    }
}
```

```

}

// 3. Determine target version
targetVersion := req.TargetVersion
if targetVersion == "" {
    if device.PreviousVersion == nil {
        return nil, ErrNoRollbackAvailable{DeviceID: deviceID}
    }
    targetVersion = *device.PreviousVersion
}

// 4. Verify rollback is possible (merge not completed)
if device.MergeCompleted {
    return nil, ErrRollbackNotAvailable{
        DeviceID: deviceID,
        Reason:    "Virtual A/B merge has completed; slot swap
rollback is not possible",
    }
}

// 5. Determine strategy
strategy := RollbackStrategy(req.Strategy)
if strategy == "" {
    strategy = StrategyInstant
}

// 6. Create rollback record
record := &RollbackRecord{
    ID:            s.idGenerator("rbk_"),
    DeviceID:      deviceID,
    FromVersion:   device.CurrentVersion,
    ToVersion:     targetVersion,
    TriggerType:   TriggerServer,
    TriggerReason: req.Reason,
    TriggeredBy:   triggeredBy,
    Strategy:      strategy,
    Status:        RollbackPending,
    MergeCompleted: device.MergeCompleted,
    CreatedAt:     time.Now(),
}

if err := s.rollbackRepo.Create(ctx, record); err != nil {
    return nil, fmt.Errorf("create rollback record: %w", err)
}

// 7. Invalidate device cache so next check-in picks up the
//      rollback
s.cache.Delete(ctx, fmt.Sprintf("device:update_check:%s",
deviceID))

// 8. Publish rollback event
s.events.Publish(ctx, eventbus.Event{
    Type:    "rollback.device.initiated",

```

```

        Payload: record,
    })

    return record, nil
}

// RollbackRollout initiates a fleet-wide rollback for all devices
// in a rollout.
func (s *RollbackService) RollbackRollout(
    ctx context.Context,
    rolloutID string,
    req RolloutRollbackRequest,
    triggeredBy *string,
) (*RolloutRollbackResponse, error) {
    // 1. Load rollout
    rollout, err := s.rolloutRepo.GetByID(ctx, rolloutID)
    if err != nil {
        return nil, fmt.Errorf("rollout lookup: %w", err)
    }

    if rollout.Status == "ROLLING_BACK" {
        return nil, ErrRolloutAlreadyRollingBack{RolloutID:
            rolloutID}
    }

    // 2. Load all devices in this rollout
    devices, err := s.deviceRepo.ListByRolloutID(ctx, rolloutID)
    if err != nil {
        return nil, fmt.Errorf("device list: %w", err)
    }

    // 3. Classify devices
    var preMerge, postMerge, offline int
    var preMergeDevices []Device
    for _, d := range devices {
        if d.Status == "offline" {
            offline++
        } else if d.MergeCompleted {
            postMerge++
        } else {
            preMerge++
            preMergeDevices = append(preMergeDevices, d)
        }
    }

    excludedPostMerge := postMerge
    if req.IncludePostMerge {
        excludedPostMerge = 0
    }

    // 4. Handle dry run
    if req.DryRun {
        return &RolloutRollbackResponse{

```

```

        RollbackID:      s.idGenerator("rbk_"),
        RolloutID:       rolloutID,
        FromVersion:     rollout.TargetVersion,
        TargetVersion:   "", // would need previous artifact
        Strategy:        req.Strategy,
        TotalDevices:    len(devices),
        PreMergeDevices:  preMerge,
        PostMergeDevices: postMerge,
        OfflineDevices:   offline,
        ExcludedPostMerge: excludedPostMerge,
        Status:          "dry_run",
        CreatedAt:       time.Now(),
    }, nil
}

// 5. Determine strategy
strategy := req.Strategy
if strategy == "" {
    strategy = "gradual"
}

// 6. Compute waves for gradual strategy
var waves []RollbackWave
targetDevices := preMergeDevices
if req.IncludePostMerge {
    // Would also include post-merge devices (requires
    downgrade artifact)
}

wavePct := req.WavePercentage
if wavePct == 0 {
    wavePct = 25
}
waveInterval := time.Duration(req.WaveIntervalSeconds) *
    time.Second
if waveInterval == 0 {
    waveInterval = 300 * time.Second
}

if strategy == StrategyGradual {
    waveSize := len(targetDevices) * wavePct / 100
    if waveSize == 0 {
        waveSize = 1
    }
    numWaves := (len(targetDevices) + waveSize - 1) / waveSize
    for i := 0; i < numWaves; i++ {
        startIdx := i * waveSize
        endIdx := startIdx + waveSize
        if endIdx > len(targetDevices) {
            endIdx = len(targetDevices)
        }
        waves = append(waves, RollbackWave{
            WaveNumber: i + 1,

```

```

        DeviceCount: endIdx - startIdx,
        StartsAt:    time.Now().Add(time.Duration(i) *
waveInterval),
    })
}
} else {
    // Instant: single wave with all devices
    waves = []RollbackWave{{
        WaveNumber: 1,
        DeviceCount: len(targetDevices),
        StartsAt:    time.Now(),
    }}
}

// 7. Pause the rollout
if err := s.rolloutRepo.UpdateStatus(ctx, rolloutID,
    "ROLLING_BACK"); err != nil {
    return nil, fmt.Errorf("pause rollout: %w", err)
}

// 8. Create rollback records for each pre-merge device
rollbackID := s.idGenerator("rbk_")
for _, device := range targetDevices {
    record := &RollbackRecord{
        ID:            s.idGenerator("rbk_"),
        DeviceID:       device.ID,
        RolloutID:      &rolloutID,
        FromVersion:    device.CurrentVersion,
        ToVersion:      *device.PreviousVersion,
        TriggerType:    TriggerServer,
        TriggerReason:  req.Reason,
        TriggeredBy:    triggeredBy,
        Strategy:       RollbackStrategy(strategy),
        Status:         RollbackPending,
        MergeCompleted: device.MergeCompleted,
        CreatedAt:      time.Now(),
    }
    if err := s.rollbackRepo.Create(ctx, record); err != nil {
        return nil, fmt.Errorf("create rollback record for
device %s: %w", device.ID, err)
    }
    // Invalidate device cache
    s.cache.Delete(ctx, fmt.Sprintf("device:update_check:%s",
device.ID))
}

// 9. Publish fleet rollback event
s.events.Publish(ctx, eventbus.Event{
    Type: "rollback.fleet.initiated",
    Payload: map[string]interface{}{
        "rollback_id": rollbackID,
        "rollout_id":  rolloutID,
        "total_devices": len(targetDevices),
    },

```

```

        "strategy":      strategy,
    },
})

return &RolloutRollbackResponse{
    RollbackID:      rollbackID,
    RolloutID:      rolloutID,
    FromVersion:      rollout.TargetVersion,
    TargetVersion:    "",
    Strategy:         strategy,
    TotalDevices:     len(devices),
    PreMergeDevices:  preMerge,
    PostMergeDevices: postMerge,
    OfflineDevices:   offline,
    ExcludedPostMerge: excludedPostMerge,
    Waves:            waves,
    Status:           "pending",
    CreatedAt:        time.Now(),
}, nil
}

// ProcessRollbackOnCheckIn handles a pending rollback when a
// device checks in.
// This is called from the update check flow when a rollback is
// pending for the device.
func (s *RollbackService) ProcessRollbackOnCheckIn(
    ctx context.Context,
    deviceID string,
) (*RollbackInstruction, error) {
    record, err := s.rollbackRepo.GetActiveByDeviceID(ctx,
        deviceID)
    if err != nil || record == nil {
        return nil, nil // No pending rollback
    }

    // Mark the rollback as in progress
    if err := s.rollbackRepo.UpdateStatus(ctx, record.ID,
        RollbackInProgress, nil); err != nil {
        return nil, fmt.Errorf("update rollback status: %w", err)
    }

    return &RollbackInstruction{
        RollbackID:      record.ID,
        FromVersion:      record.FromVersion,
        ToVersion:        record.ToVersion,
        TriggerType:      string(record.TriggerType),
        Reason:           record.TriggerReason,
    }, nil
}

// CompleteRollback marks a rollback as completed after the device
// confirms it.
func (s *RollbackService) CompleteRollback(

```

```

    ctx context.Context,
    rollbackID string,
    healthCheckData map[string]interface{},
) error {
    now := time.Now()
    if err := s.rollbackRepo.UpdateStatus(ctx, rollbackID,
        RollbackCompleted, &now); err != nil {
        return fmt.Errorf("update rollback status: %w", err)
    }

    // Get the rollback record to update the device version
    record, err := s.rollbackRepo.GetByID(ctx, rollbackID)
    if err != nil {
        return fmt.Errorf("get rollback record: %w", err)
    }

    // Update the device's current version to the rollback target
    previousSlot := "_a"
    if record.FromVersion != "" {
        previousSlot = "_b" // Simplified; real impl would track
        slot
    }
    if err := s.deviceRepo.UpdateVersion(ctx, record.DeviceID,
        record.ToVersion, previousSlot); err != nil {
        return fmt.Errorf("update device version: %w", err)
    }

    // Publish completion event
    s.events.Publish(ctx, eventbus.Event{
        Type: "rollback.completed",
        Payload: map[string]interface{}{
            "rollback_id":      rollbackID,
            "device_id":        record.DeviceID,
            "from_version":     record.FromVersion,
            "to_version":       record.ToVersion,
            "health_check_data": healthCheckData,
        },
    })

    return nil
}

// RollbackInstruction is sent to the device during an update
// check
// when a rollback is pending.
type RollbackInstruction struct {
    RollbackID string `json:"rollback_id"`
    FromVersion string `json:"from_version"`
    ToVersion string `json:"to_version"`
    TriggerType string `json:"trigger_type"`
    Reason string `json:"reason"`
}

```

```
// Error Types
//
```

```

type ErrRollbackAlreadyInProgress struct {
    DeviceID string
}

func (e ErrRollbackAlreadyInProgress) Error() string {
    return fmt.Sprintf("rollback already in progress for device %s", e.DeviceID)
}

type ErrNoRollbackAvailable struct {
    DeviceID string
}

func (e ErrNoRollbackAvailable) Error() string {
    return fmt.Sprintf("no rollback available for device %s", e.DeviceID)
}

type ErrRollbackNotAvailable struct {
    DeviceID string
    Reason   string
}

func (e ErrRollbackNotAvailable) Error() string {
    return fmt.Sprintf("rollback not available for device %s: %s", e.DeviceID, e.Reason)
}

type ErrRolloutAlreadyRollingBack struct {
    RolloutID string
}

func (e ErrRolloutAlreadyRollingBack) Error() string {
    return fmt.Sprintf("rollout %s is already rolling back", e.RolloutID)
}

func generateID(prefix string) string {
    // In production, this uses a ULID or similar.
    return fmt.Sprintf("%s%d", prefix, time.Now().UnixNano())
}

```

10. Reference Implementation: Kotlin Android Client Rollback

The following Kotlin code implements the client-side rollback system, integrating with the Android Boot Control HAL and the existing Helix OTA client architecture.

```

package com.helix.ota.client.rollback

import android.content.Context
import android.os.IUpdateEngineCallback
import android.util.Log
import com.helix.ota.client.engine.BootControlHelper
import com.helix.ota.client.engine.UpdateEngineProxy
import com.helix.ota.client.service.ReportingService
import com.helix.ota.client.state.OtaEvent
import com.helix.ota.client.state.OtaState
import com.helix.ota.client.state.OtaStateMachine
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.io.File
import java.util.concurrent.TimeUnit
import javax.inject.Inject
import javax.inject.Singleton

// -----
// Domain Types
// -----

enum class RollbackTrigger(val type: String) {
    AUTOMATIC("automatic"),
    SERVER("server"),
    USER("user")
}

enum class RollbackStatus {
    IDLE,
    CHECKING,
    INITIATED,
    SWITCHING_SLOT,
    REBOOTING,
    VERIFYING,
    COMPLETED,
    FAILED
}

data class RollbackInfo(
    val rollbackId: String,
    val fromVersion: String,
    val toVersion: String,
    val trigger: RollbackTrigger,
    val reason: String
)

data class HealthCheckResult(
    val bootTimeMs: Long,
    val bootTimeBaselineMs: Long,

```

```

        val bootTimeExceeded: Boolean,
        val criticalServicesOk: Boolean,
        val storageAccessible: Boolean,
        val networkAvailable: Boolean,
        val peripheralStatusOk: Boolean,
        val shouldRollback: Boolean
    )

// -----
// Rollback Manager
// -----

/**
 * Central component for managing rollback operations on the
 * device.
 * Integrates with the Boot Control HAL, update_engine, and the
 * Helix OTA server to execute and report rollback operations.
 */
@Singleton
class RollbackManager @Inject constructor(
    @ApplicationContext private val context: Context,
    private val bootControl: BootControlHelper,
    private val updateEngine: UpdateEngineProxy,
    private val reportingService: ReportingService,
    private val stateMachine: OtaStateMachine,
    private val prefs: OtaPreferences,
    private val healthChecker: HealthCheckService
) {
    private val scope = CoroutineScope(Dispatchers.IO +
        SupervisorJob())

    private val _rollbackState =
        MutableStateFlow(RollbackStatus.IDLE)
    val rollbackState: StateFlow<RollbackStatus> =
        _rollbackState.asStateFlow()

    companion object {
        private const val TAG = "RollbackManager"
        private const val ROLLBACK_WINDOW_DAYS = 7L
        private const val MAX_BOOT_RETRIES = 3
        private const val HEALTH_CHECK_TIMEOUT_MS = 60_000L
        private const val BOOT_TIME_THRESHOLD_RATIO = 1.5 // 150%
            of baseline
    }
}

// -----
// Boot Failure Rollback Detection
// -----

/**
 * Called by BootCompletedReceiver after every boot.
 * Detects if an automatic boot-time rollback occurred by
 * checking

```

```

    * whether the current slot differs from the expected post-
      update slot.
    */
fun detectBootRollback() {
    val currentSlot = bootControl.getCurrentSlot()
    val expectedSlot = prefs.expectedPostUpdateSlot

    // If we have a pending update and we're NOT in the
    // expected slot,
    // a boot-time rollback occurred
    if (expectedSlot != -1 && currentSlot != expectedSlot) {
        val fromVersion = prefs.pendingUpdateVersion
        val toVersion = prefs.installedVersion

        Log.w(TAG, "Boot-time rollback detected! " +
            "Expected slot $expectedSlot, booted into slot
            $currentSlot. " +
            "Rolling back from $fromVersion to $toVersion")

        scope.launch {
            reportAutomaticRollback(fromVersion, toVersion,
            currentSlot)
        }

        // Update local state
        prefs.installedVersion = toVersion
        prefs.pendingUpdateVersion = ""
        prefs.expectedPostUpdateSlot = -1
        stateMachine.transition(OtaEvent.Failed("Automatic
        rollback: boot failure"))

        // Do NOT mark boot successful on the new slot – it
        // failed
    } else if (expectedSlot != -1 && currentSlot ==
    expectedSlot) {
        // Booted into the expected (new) slot – run health
        // checks
        Log.i(TAG, "Booted into new slot
        $currentSlot. Running health checks...")
        scope.launch {
            runPostUpdateHealthChecks()
        }
    }
}

/**
 * Reports an automatic (boot-time) rollback to the Helix OTA
 * server.
 */
private suspend fun reportAutomaticRollback(
    fromVersion: String,
    toVersion: String,
    rollbackSlot: Int
) {

```

```

_rollbackState.value = RollbackStatus.COMPLETED

reportingService.reportRollback(
    triggerType = RollbackTrigger.AUTOMATIC.type,
    fromVersion = fromVersion,
    toVersion = toVersion,
    fromSlot = prefs.expectedPostUpdateSlot,
    toSlot = rollbackSlot,
    healthCheckResults = null, // No health check data –
    // boot failed
    reason = "Automatic rollback: device failed to boot
    into new slot after $MAX_BOOT_RETRIES attempts"
)
}

// -----
// Health Check & Automatic Rollback
// -----

/**
 * Runs post-update health checks. If any critical check
 * fails,
 * automatically initiates a rollback before
 * markBootSuccessful()
 * is called (preventing the Virtual A/B merge from starting).
 */
private suspend fun runPostUpdateHealthChecks() {
    _rollbackState.value = RollbackStatus.CHECKING

    try {
        withTimeout(HEALTH_CHECK_TIMEOUT_MS) {
            val result = healthChecker.runChecks()

            if (result.shouldRollback) {
                Log.w(TAG, "Health check failed – initiating
                automatic rollback. " +
                    "Boot time: ${result.bootTimeMs}ms " +
                    "(baseline: ${result.bootTimeBaselineMs}
                ms), " +
                    "Critical services OK: $
                {result.criticalServicesOk}, " +
                    "Storage accessible: $
                {result.storageAccessible}")

                initiateRollback(
                    trigger = RollbackTrigger.AUTOMATIC,
                    reason =
                    buildHealthCheckFailureReason(result),
                    healthCheckResults = result
                )
            } else {
                Log.i(TAG,
                "Health checks passed. Marking boot successful.")
                bootControl.markBootSuccessful()
            }
        }
    }
}

```

```

        prefs.bootTimeBaselineMs = result.bootTimeMs
        stateMachine.transition(OtaEvent.Succeeded)

reportingService.reportStatus("update_succeeded")
    }
}
} catch (e: TimeoutCancellationException) {
    Log.e(TAG, "Health check timed out after $
{HEALTH_CHECK_TIMEOUT_MS}ms - rolling back")
    initiateRollback(
        trigger = RollbackTrigger.AUTOMATIC,
        reason = "Health check timed out after $
{HEALTH_CHECK_TIMEOUT_MS}ms",
        healthCheckResults = null
    )
} catch (e: Exception) {
    Log.e(TAG, "Health check error", e)
    // On unexpected error, be conservative and mark boot
    successful
    // to avoid getting stuck in a rollback loop
    bootControl.markBootSuccessful()
    stateMachine.transition(OtaEvent.Succeeded)
}
}

private fun buildHealthCheckFailureReason(result:
HealthCheckResult): String {
    val reasons = mutableListOf<String>()
    if (result.bootTimeExceeded) {
        reasons.add("Boot time ${result.bootTimeMs}ms exceeds
" +
            "baseline ${result.bootTimeBaselineMs}ms by >50%")
    }
    if (!result.criticalServicesOk) {
        reasons.add("Critical system services failed health
check")
    }
    if (!result.storageAccessible) {
        reasons.add("Storage access verification failed")
    }
    return "Health check failure: ${reasons.joinToString(";
")}"}"
}

// -----
// Server-Triggered Rollback
// -----

/**
 * Processes a rollback instruction received from the Helix
 * OTA server
 * during an update check. This is called when the server
 * includes a
 * rollback_command in the update check response.

```

```

    */
fun processServerRollback(instruction: RollbackInstruction) {
    Log.i(TAG, "Received server rollback command: $
        {instruction.reason}")

    scope.launch {
        initiateRollback(
            trigger = RollbackTrigger.SERVER,
            reason = instruction.reason,
            rollbackId = instruction.rollbackId,
            healthCheckResults = null
        )
    }
}

// =====
// User-Initiated Rollback
// =====

/**
 * Checks whether a user-initiated rollback is currently
 * available.
 */
fun isRollbackAvailable(): Boolean {
    if (_rollbackState.value != RollbackStatus.IDLE) return
        false

    val previousSlot = getPreviousSlot() ?: return false
    if (!bootControl.isSlotBootable(previousSlot)) return
        false
    if (bootControl.isMergePending()) return false // Merge
        already started

    // Check rollback window
    val updateTime = prefs.lastUpdateTime
    if (updateTime > 0) {
        val daysSinceUpdate = TimeUnit.MILLISECONDS.toDays(
            System.currentTimeMillis() - updateTime
        )
        if (daysSinceUpdate > ROLLBACK_WINDOW_DAYS) return
            false
    }

    return true
}

/**
 * Returns information about the available rollback for
 * display in Settings.
 */
fun getRollbackInfo(): RollbackInfo? {
    if (!isRollbackAvailable()) return null

    return RollbackInfo(

```

```

        rollbackId = "",
        fromVersion = prefs.installedVersion,
        toVersion = prefs.previousVersion,
        trigger = RollbackTrigger.USER,
        reason = "User-initiated rollback from Settings"
    )
}

/**
 * Executes a user-initiated rollback. Called from the
 * Settings UI.
 */
suspend fun executeUserRollback(): Result<Unit> {
    if (!isRollbackAvailable()) {
        return Result.failure(IllegalStateException("Rollback
        is not available"))
    }

    return try {
        initiateRollback(
            trigger = RollbackTrigger.USER,
            reason = "User-initiated rollback from Settings",
            healthCheckResults = null
        )
        Result.success(Unit)
    } catch (e: Exception) {
        Log.e(TAG, "User rollback failed", e)
        Result.failure(e)
    }
}

// =====
// Core Rollback Execution
// =====

/**
 * Core rollback execution. Switches the active boot slot to
 * the
 * previous slot and reboots the device.
 */
private suspend fun initiateRollback(
    trigger: RollbackTrigger,
    reason: String,
    rollbackId: String? = null,
    healthCheckResults: HealthCheckResult?
) {
    _rollbackState.value = RollbackStatus.INITIATED

    val previousSlot = getPreviousSlot()
    if (previousSlot == null) {
        Log.e(TAG, "Cannot rollback: no previous slot
        available")
        _rollbackState.value = RollbackStatus.FAILED
    }
}

```

```

        return
    }

    if (!bootControl.isSlotBootable(previousSlot)) {
        Log.e(TAG, "Cannot rollback: previous slot
$previousSlot is not bootable")
        _rollbackState.value = RollbackStatus.FAILED
        return
    }

    val currentVersion = prefs.installedVersion
    val targetVersion = prefs.previousVersion

    try {
        // Step 1: Report rollback initiation to server
        _rollbackState.value = RollbackStatus.SWITCHING_SLOT

        // Step 2: Switch active boot slot
        Log.i(TAG, "Switching active boot slot to
$previousSlot for rollback")
        bootControl.setActiveBootSlot(previousSlot)

        // Step 3: Report rollback to server before reboot
        reportingService.reportRollback(
            triggerType = trigger.type,
            fromVersion = currentVersion,
            toVersion = targetVersion,
            fromSlot = bootControl.getCurrentSlot(),
            toSlot = previousSlot,
            healthCheckResults = healthCheckResults,
            rollbackId = rollbackId,
            reason = reason
        )

        // Step 4: Update local state
        prefs.installedVersion = targetVersion
        prefs.previousVersion = currentVersion
        prefs.expectedPostUpdateSlot = previousSlot

        // Step 5: Snapshot config for migration
        snapshotConfiguration()

        // Step 6: Reboot
        _rollbackState.value = RollbackStatus.REBOOTING
        Log.i(TAG, "Rebooting device for rollback to
$targetVersion")
        rebootDevice()
    } catch (e: Exception) {
        Log.e(TAG, "Rollback failed during execution", e)
        _rollbackState.value = RollbackStatus.FAILED

        // Attempt to report failure
    }

```

```

        try {
            reportingService.reportStatus("rollback_failed",
e.message)
        } catch (reportErr: Exception) {
            Log.e(TAG, "Failed to report rollback failure",
reportErr)
        }
    }
}

// -----
// Helpers
// -----

private fun getPreviousSlot(): Int? {
    val currentSlot = bootControl.getCurrentSlot()
    val numSlots = bootControl.getNumberSlots()
    if (numSlots != 2) return null

    // Previous slot is the other slot (0 ↔ 1)
    return if (currentSlot == 0) 1 else 0
}

private fun snapshotConfiguration() {
    val snapshotDir = File(context.filesDir,
"rollback_config_snapshot")
    snapshotDir.mkdirs()

    val pathsToSnapshot = listOf(
        "/data/system/packages.xml",
        "/data/misc/wifi/wpa_supplicant.conf"
    )

    for (path in pathsToSnapshot) {
        try {
            val source = File(path)
            if (source.exists()) {
                source.copyTo(File(snapshotDir, source.name),
overwrite = true)
            }
        } catch (e: Exception) {
            Log.w(TAG, "Failed to snapshot $path", e)
        }
    }
}

private fun rebootDevice() {
    // Use PowerManager to reboot
    val powerManager =
context.getSystemService(Context.POWER_SERVICE)
        as android.os.PowerManager
    powerManager.reboot("helix-ota-rollback")
}

```

```

}

// -----
// Health Check Service
// -----

/**
 * Performs post-update health checks before marking the boot as
 * successful.
 * If critical checks fail, triggers an automatic rollback.
 */
@Singleton
class HealthCheckService @Inject constructor(
    @ApplicationContext private val context: Context,
    private val prefs: OtaPreferences
) {
    companion object {
        private const val TAG = "HealthCheckService"
        private const val BOOT_TIME_THRESHOLD_RATIO = 1.5
    }

    suspend fun runChecks(): HealthCheckResult {
        val bootTimeMs = measureBootTime()
        val baselineMs = prefs.bootTimeBaselineMs
        val bootTimeExceeded = baselineMs > 0 &&
            bootTimeMs > (baselineMs * BOOT_TIME_THRESHOLD_RATIO)

        val criticalServicesOk = checkCriticalServices()
        val storageAccessible = checkStorageAccess()
        val networkAvailable = checkNetworkConnectivity()
        val peripheralStatusOk = checkPeripherals()

        // Determine if rollback is needed based on critical
        // failures
        val shouldRollback = bootTimeExceeded || !
            criticalServicesOk || !storageAccessible

        return HealthCheckResult(
            bootTimeMs = bootTimeMs,
            bootTimeBaselineMs = baselineMs,
            bootTimeExceeded = bootTimeExceeded,
            criticalServicesOk = criticalServicesOk,
            storageAccessible = storageAccessible,
            networkAvailable = networkAvailable,
            peripheralStatusOk = peripheralStatusOk,
            shouldRollback = shouldRollback
        )
    }

    private fun measureBootTime(): Long {
        // Read boot time from system properties
        return try {

```

```

        val bootTime =
            android.os.SystemClock.elapsedRealtime()
            val uptime = android.os.SystemClock.uptimeMillis()
            // Boot completed time is approximately
            elapsedRealtime - uptime
            bootTime
    } catch (e: Exception) {
        Log.e(TAG, "Failed to measure boot time", e)
        Long.MAX_VALUE // Fail-safe: assume excessive boot
        time
    }
}

private fun checkCriticalServices(): Boolean {
    return try {
        val serviceManager =
            Class.forName("android.os.ServiceManager")
        val getService =
            serviceManager.getMethod("getService", String::class.java)

        val criticalServices = listOf("activity", "package",
            "window", "mount")
        for (serviceName in criticalServices) {
            val binder = getService.invoke(null, serviceName)
            as? android.os.IBinder
            if (binder == null || !binder.isBinderAlive) {
                Log.w(TAG, "Critical service $serviceName is
                not alive")
                return false
            }
        }
        true
    } catch (e: Exception) {
        Log.e(TAG, "Failed to check critical services", e)
        false
    }
}

private fun checkStorageAccess(): Boolean {
    return try {
        val testFile = File(context.filesDir,
            ".health_check_test")
        testFile.writeText("health_check_$
        {System.currentTimeMillis()}")
        val content = testFile.readText()
        testFile.delete()
        content.startsWith("health_check_")
    } catch (e: Exception) {
        Log.e(TAG, "Storage access check failed", e)
        false
    }
}

private fun checkNetworkConnectivity(): Boolean {

```

```

        // Network check is advisory – failure does not trigger
        rollback
        return try {
            val cm =
                context.getSystemService(Context.CONNECTIVITY_SERVICE)
                    as android.net.ConnectivityManager
            val network = cm.activeNetwork
            network != null
        } catch (e: Exception) {
            false
        }
    }

    private fun checkPeripherals(): Boolean {
        // RK3588-specific peripheral checks (advisory only)
        return try {
            val gpuNode = File("/dev/mali0")
            gpuNode.exists()
        } catch (e: Exception) {
            false
        }
    }
}

// -----
// Boot Control Helper (Extended for Rollback)
// -----

/**
 * Wrapper around the Android Boot Control HAL, extended with
 * rollback-specific operations.
 */
@Singleton
class BootControlHelper @Inject constructor(
    @ApplicationContext private val context: Context
) {
    companion object {
        private const val TAG = "BootControlHelper"
    }

    fun getCurrentSlot(): Int {
        // Access Boot Control HAL via HIDL/AIDL
        return try {
            val service =
                android.hardware.boot.BootControl.Stub.asInterface(
                    android.os.ServiceManager.waitForService("android.hardware.boot.BootControl")
                )
            service.currentSlot
        } catch (e: Exception) {
            Log.e(TAG, "Failed to get current slot", e)
            0 // Default to slot 0
        }
    }
}

```

```

    }
}

fun getNumberSlots(): Int {
    return try {
        val service =
            android.hardware.boot.ILogControl.Stub.asInterface(

                android.os.ServiceManager.waitForService("android.hardware.boot.ILogControl
                default")
            )
        service.numberSlots
    } catch (e: Exception) {
        Log.e(TAG, "Failed to get number of slots", e)
        0
    }
}

fun setActiveBootSlot(slot: Int) {
    try {
        val service =
            android.hardware.boot.ILogControl.Stub.asInterface(

                android.os.ServiceManager.waitForService("android.hardware.boot.ILogControl
                default")
            )
        service.setActiveBootSlot(slot)
        Log.i(TAG, "Set active boot slot to $slot")
    } catch (e: Exception) {
        Log.e(TAG, "Failed to set active boot slot to $slot",
            e)
        throw BootControlException("Failed to set active boot
            slot: ${e.message}")
    }
}

fun markBootSuccessful() {
    try {
        val service =
            android.hardware.boot.ILogControl.Stub.asInterface(

                android.os.ServiceManager.waitForService("android.hardware.boot.ILogControl
                default")
            )
        service.markBootSuccessful()
        Log.i(TAG, "Marked current boot as successful")
    } catch (e: Exception) {
        Log.e(TAG, "Failed to mark boot as successful", e)
        throw BootControlException("Failed to mark boot
            successful: ${e.message}")
    }
}

fun isSlotBootable(slot: Int): Boolean {

```

```

        return try {
            val service =
                android.hardware.boot.ILogControl.Stub.asInterface(

                android.os.ServiceManager.waitForService("android.hardware.boot.ILogControl
                default")
            )
            service.isSlotBootable(slot) == 1
        } catch (e: Exception) {
            Log.e(TAG, "Failed to check if slot $slot is
            bootable", e)
            false
        }
    }

    fun isMergePending(): Boolean {
        // Check if Virtual A/B merge is pending or in progress
        return try {
            val snapshotManager =
                android.os.snapshot.ISnapshotManager.Stub.asInterface(

                android.os.ServiceManager.waitForService("android.os.snapshot.ISnapshotMan
                default")
            )
            val mergeState = snapshotManager.snapshotMergeState
            mergeState != android.os.snapshot.MergeState.NONE &&
                mergeState !=
                android.os.snapshot.MergeState.COMPLETED
        } catch (e: Exception) {
            // If we can't determine merge state, assume merge is
            NOT pending
            // (conservative approach for rollback availability)
            Log.w(TAG,
                "Cannot determine merge state, assuming not pending", e)
            false
        }
    }
}

class BootControlException(message: String) : Exception(message)

// -----
// Rollback Instruction (from server)
// -----

data class RollbackInstruction(
    val rollbackId: String,
    val fromVersion: String,
    val toVersion: String,
    val triggerType: String,
    val reason: String
)

// -----

```

```
// Preferences Extension for Rollback State
// -----

/**
 * Extended OtaPreferences with rollback-specific fields.
 * These are persisted in SharedPreferences and survive reboots.
 */
interface OtaPreferences {
    var installedVersion: String
    var previousVersion: String
    var pendingUpdateVersion: String
    var expectedPostUpdateSlot: Int
    var bootTimeBaselineMs: Long
    var lastUpdateTime: Long
    var bootCountSinceUpdate: Int
    var cachedUpdateInfo: UpdateInfo?
    var otaZipPath: String
    var deviceId: String
}

```

Appendix A: Database Schema Additions

The following tables are added to the PostgreSQL database to support rollback:

```
-- Rollback audit log (append-only)
CREATE TABLE rollback_audit_log (
    id                VARCHAR(32) PRIMARY KEY,
    device_id         VARCHAR(32) NOT NULL REFERENCES devices(id),
    rollout_id        VARCHAR(32) REFERENCES rollouts(id),
    from_version      VARCHAR(32) NOT NULL,
    to_version        VARCHAR(32) NOT NULL,
    trigger_type      VARCHAR(32) NOT NULL CHECK (trigger_type IN
        ('automatic', 'server', 'user')),
    trigger_reason    TEXT NOT NULL,
    triggered_by      VARCHAR(32),           -- user_id for server/
        user triggers
    strategy          VARCHAR(32) NOT NULL DEFAULT 'instant',
    status            VARCHAR(32) NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'in_progress',
            'completed', 'failed')),
    merge_completed   BOOLEAN NOT NULL DEFAULT FALSE,
    health_check_data JSONB DEFAULT '{}',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at      TIMESTAMPTZ
);

CREATE INDEX idx_rollback_device_id ON
    rollback_audit_log(device_id);
CREATE INDEX idx_rollback_rollout_id ON
    rollback_audit_log(rollout_id);
CREATE INDEX idx_rollback_status ON rollback_audit_log(status);

```

```

CREATE INDEX idx_rollback_created_at ON
    rollback_audit_log(created_at DESC);

-- Device table additions
ALTER TABLE devices ADD COLUMN IF NOT EXISTS previous_version
    VARCHAR(32);
ALTER TABLE devices ADD COLUMN IF NOT EXISTS merge_completed
    BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS last_rollback_at
    TIMESTAMPTZ;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS rollback_count
    INTEGER NOT NULL DEFAULT 0;

-- Rollout status enum extension
ALTER TABLE rollouts ADD CONSTRAINT chk_rollout_status
    CHECK (status IN ('ACTIVE', 'PAUSED', 'COMPLETED',
        'ROLLED_BACK', 'ROLLING_BACK'));

```

Appendix B: Configuration Reference

Configuration Key	Default	Description
rollback.window_days	7	Days after update during which user-initiated rollback is available
rollback.boot_retry_count	3	Number of boot attempts before automatic fallback
rollback.health_check_timeout_ms	60000	Maximum time for post-update health checks
rollback.boot_time_threshold_ratio	1.5	Boot time ratio (vs baseline) that triggers rollback
rollback.auto_rollback_on_health_failure	true	Automatically rollback when health checks fail
rollback.server_trigger_enabled	true	Allow server-triggered rollbacks
rollback.gradual_wave_percentage	25	Default wave size for gradual fleet rollbacks
rollback.gradual_wave_interval_seconds	300	

Configuration Key	Default	Description
		Default interval between gradual rollback waves
rollback.max_concurrent_fleet_rollbacks	1	Maximum number of concurrent fleet rollback operations
rollback.config_snapshot_enabled	true	Snapshot system config before update for rollback migration
rollback.audit_retention_days	365	Days to retain rollback audit records

Appendix C: Event Bus Events

Event Type	Payload	Consumers
rollback.device.initiated	RollbackRecord	Notification service, telemetry aggregator
rollback.fleet.initiated	FleetRollbackPayload	Notification service, dashboard
rollback.completed	RollbackRecord	Device service (version update), telemetry, notification
rollback.failed	RollbackRecord	Notification service, alert dispatcher
rollback.boot_detected	BootRollbackPayload	Telemetry, device service